

FAREGLIDE

Free-first airfare intelligence platform

SYSTEM DESIGN & DATA-SOURCE DECISION

Version 1.3 | August 2026

Product goal

Help people explore destinations on a dark terrain globe, save routes they care about, build a reusable price history, and receive high-signal alerts when fares move sharply.

Design principle

FareGlide is a tracking and decision-support product, not a ticketing agency. Live bookable prices come from a provider; historical trends come from compact observations FareGlide collects over time.

Recommendation

Use SerpApi as the primary structured collector and a guarded local Google Flights adapter as fallback and occasional validation. Cache every observation, keep sources distinct, and require an authorized provider review before public deployment.

Included

- C4 system context plus formal UML component, sequence, class, deployment, and state diagrams
- Scrape Badger, Kiwi Tequila, Aviationstack, and OpenSky assessment
- Core data model and shared-search strategy
- Alert pipeline, free-tier guardrails, risks, and phased delivery

1. Data-source decision

The sources solve different problems. Only a fare-search source can populate price observations; flight-status sources enrich the experience after a route or flight is known.

Source	Best use	Free reality	Decision
SerpApi Google Flights	Structured fares, itineraries, filters, price insights	250 successful searches/month on current free tier	Primary local collector; conserve quota adaptively
Google Flights - local adapter	Cash fares, itineraries, stops, airline, emissions	No API fee, but technically fragile and subject to Google's terms	Local-only guarded experiment; never the only provider
Scrape Badger	Managed Google Flights extraction	Trial credits; paid at sustained usage	Alternative experiment behind the same adapter boundary
Kiwi.com Tequila	Fare search and virtual interlining	New partnerships are invitation-only	Do not plan MVP around access
Aviationstack	Flight status, schedules, routes, gates	100 requests/month; personal license; fare prices are not its purpose	Optional enrichment, not pricing
OpenSky Network	Live aircraft positions and tracks	Limited public API with OAuth and usage policies	Optional live-map layer, not pricing
Skyscanner	Live and indicative fares plus deeplinks	Partnership application; commercial terms are agreement-specific	Preferred official provider if accepted
Amadeus Self-Service	Official fare search for prototyping	Free monthly quota; incomplete airline coverage	Fallback official adapter

Google Flights and Scrape Badger assessment

- Strong functional fit for a private prototype: route searches can yield total price, duration, stops, carrier, and itinerary details.
- Local and noncommercial use lowers practical exposure but does not override Google terms or make automation automatically authorized.
- The local adapter is action-scoped: adding a route, a permitted manual refresh, or the adaptive scheduler can request data; map panning cannot.
- Never log in, solve or bypass CAPTCHAs, rotate proxies, evade blocks, or continue after an explicit denial. A block disables the adapter.
- Use only behind a provider adapter, with cache-first reads, a kill switch, hard request budgets, jitter, backoff, and no raw HTML retention.

Tandem collection policy

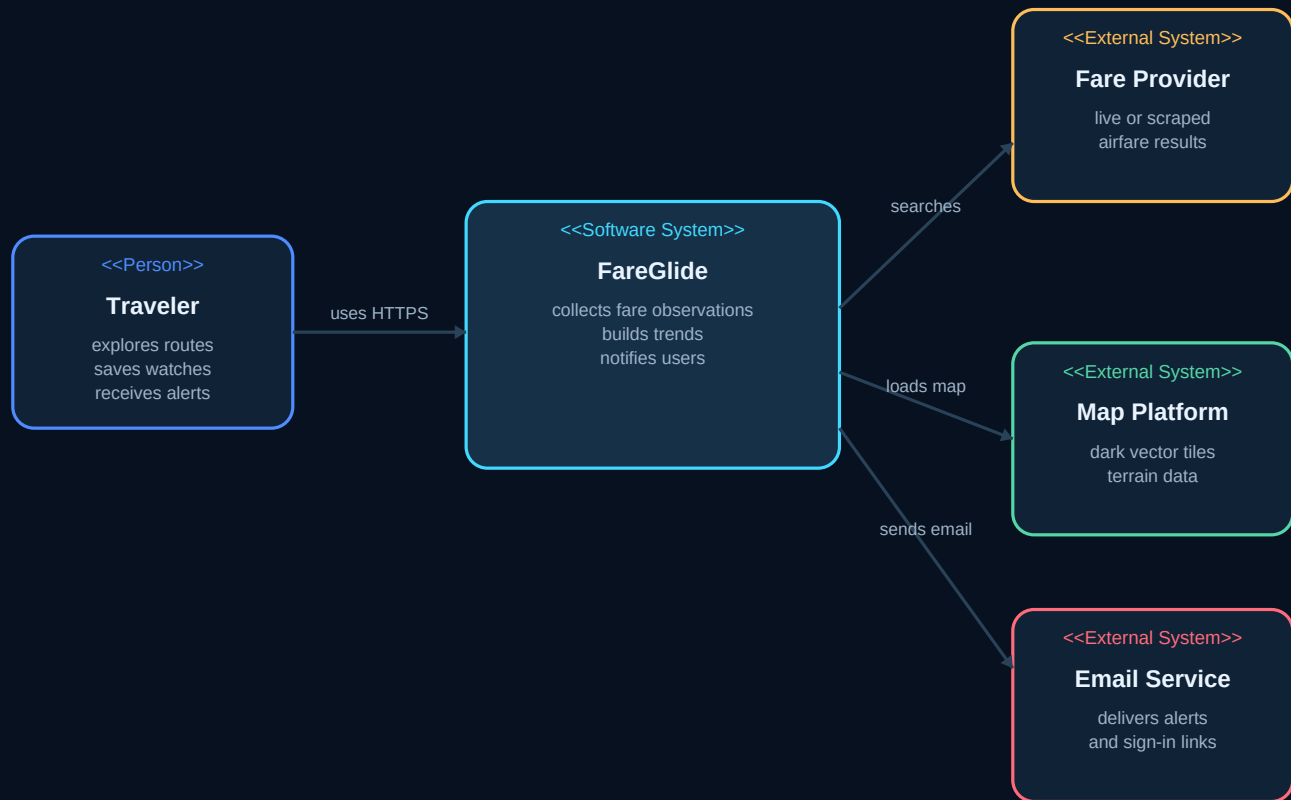
SerpApi is authoritative for routine collection. The local scraper runs only as a fallback or sampled validation source, never automatically beside every SerpApi request. Observations retain their provider identity and are not blended into an invented fare.

Kiwi, Aviationstack, and OpenSky

Kiwi Tequila can no longer be treated as a public developer API; Kiwi states that new Tequila partnerships are invitation-only. Aviationstack and OpenSky provide operational aviation data, not historical airfare quotes. They can later power flight-status cards, route metadata, aircraft positions, and disruption context without entering the price-calculation path.

2. C4 system context

The C4 context view establishes FareGlide's system boundary, primary user, and external dependencies before the design moves into UML detail.

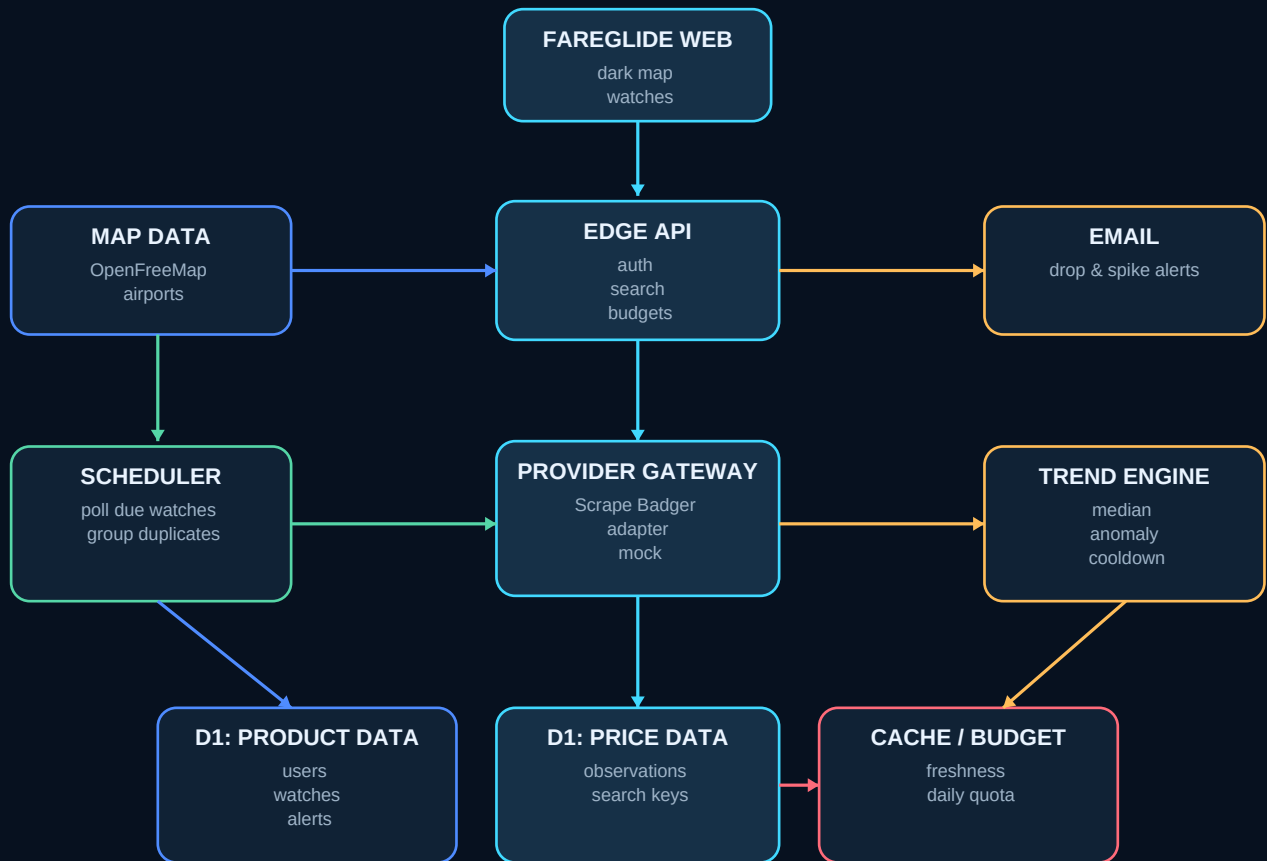


Context interpretation

- The traveler interacts only with FareGlide; provider credentials never reach the browser.
- Fare providers supply current search results, while FareGlide owns watches, observations, trends, and alerts.
- Map and notification platforms are replaceable external systems behind narrow interfaces.

3. UML component diagram

Components show runtime responsibilities and dependency direction. Database boxes represent logical components even when they share one physical D1 deployment.



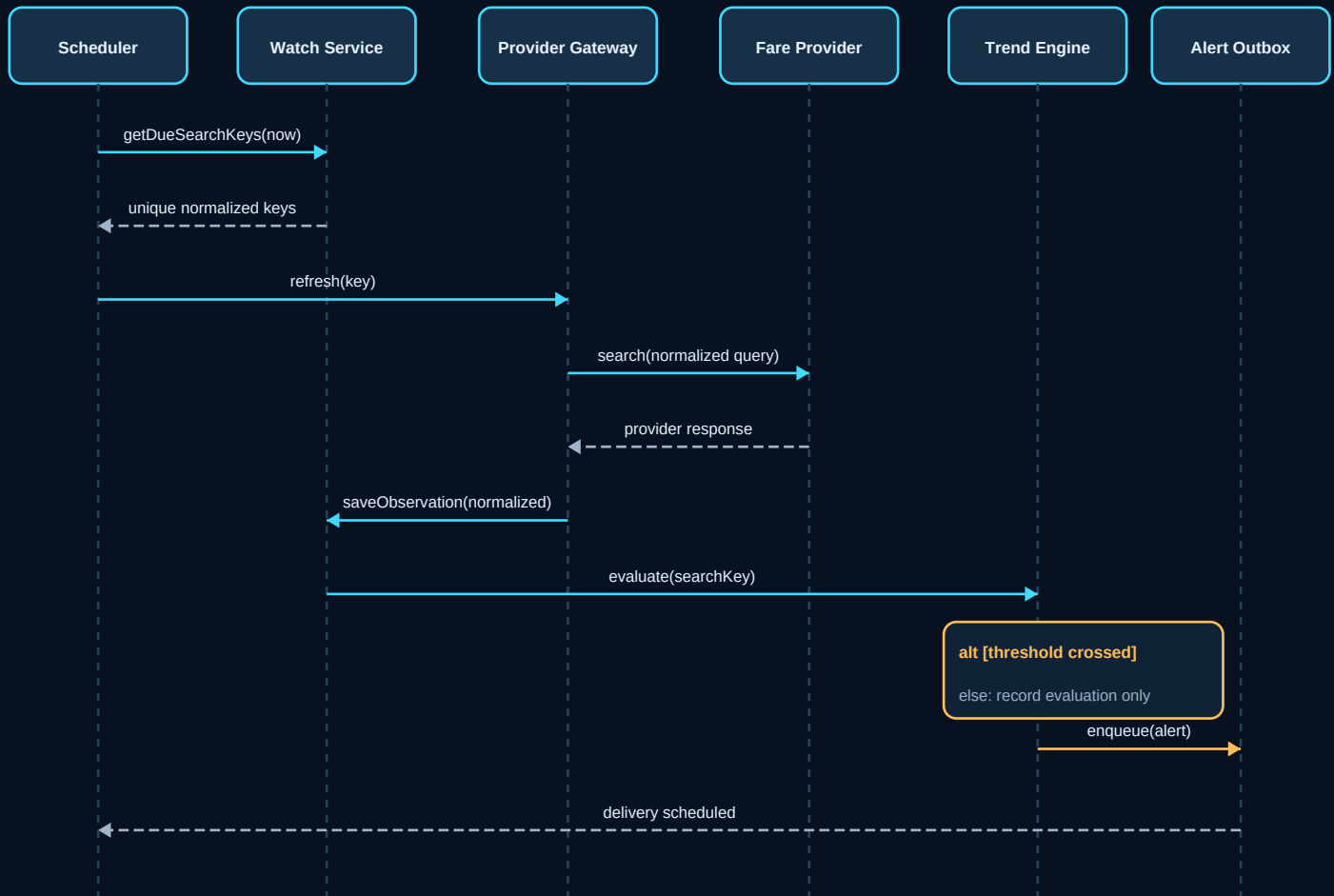
Key boundaries

The browser never talks directly to an airfare provider or receives provider credentials.

- All provider responses are normalized into FareGlide's own observation format.
- Product records and price observations are stored separately so retention policies can differ.
- The scheduler reads only watches that are due, then groups identical search keys before fetching.
- The trend engine is deterministic and explainable; no AI prediction is required for the MVP.

4. UML sequence - scheduled refresh

The sequence below captures one scheduled refresh cycle. The provider gateway isolates vendor-specific requests and response formats.

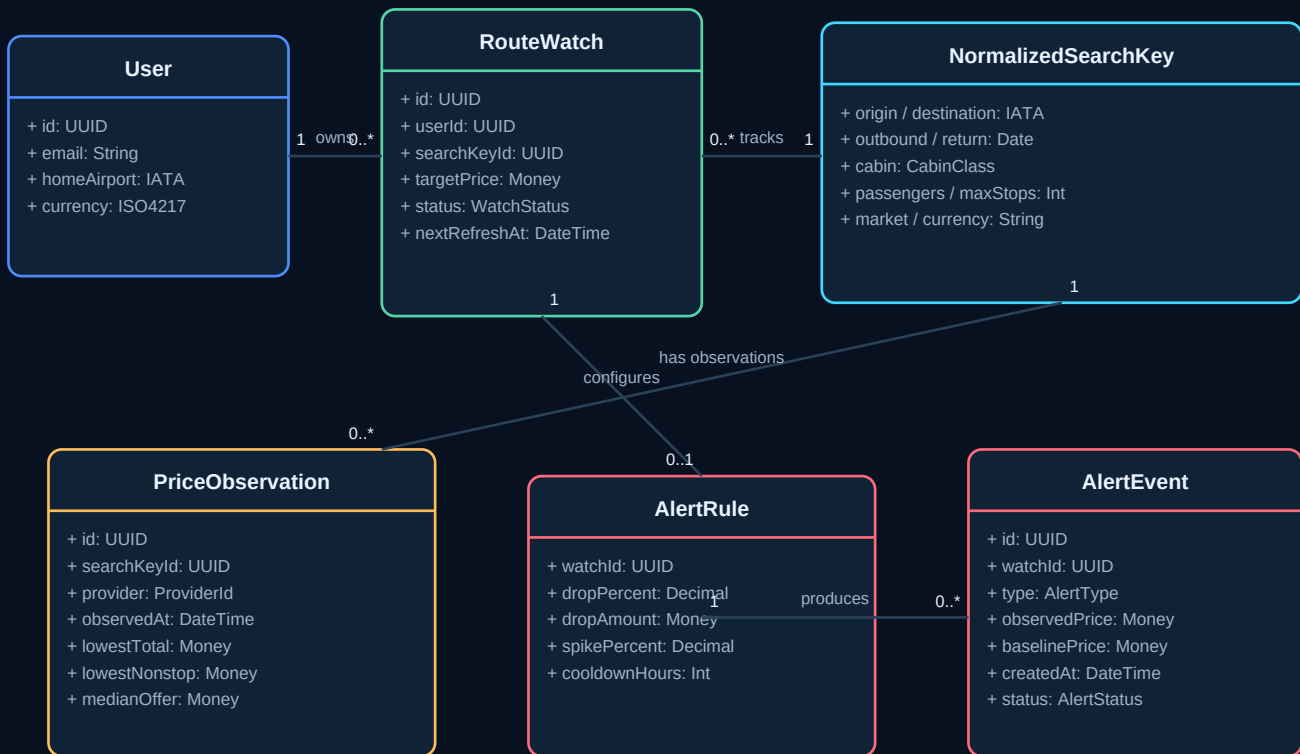


Sequence guarantee

A provider failure must not delete history or trigger a false alert. The refresh is recorded as an error, the watch remains active, and the scheduler applies backoff before trying again.

5. UML class diagram

The conceptual domain model separates user-owned configuration from reusable, non-personal fare observations.

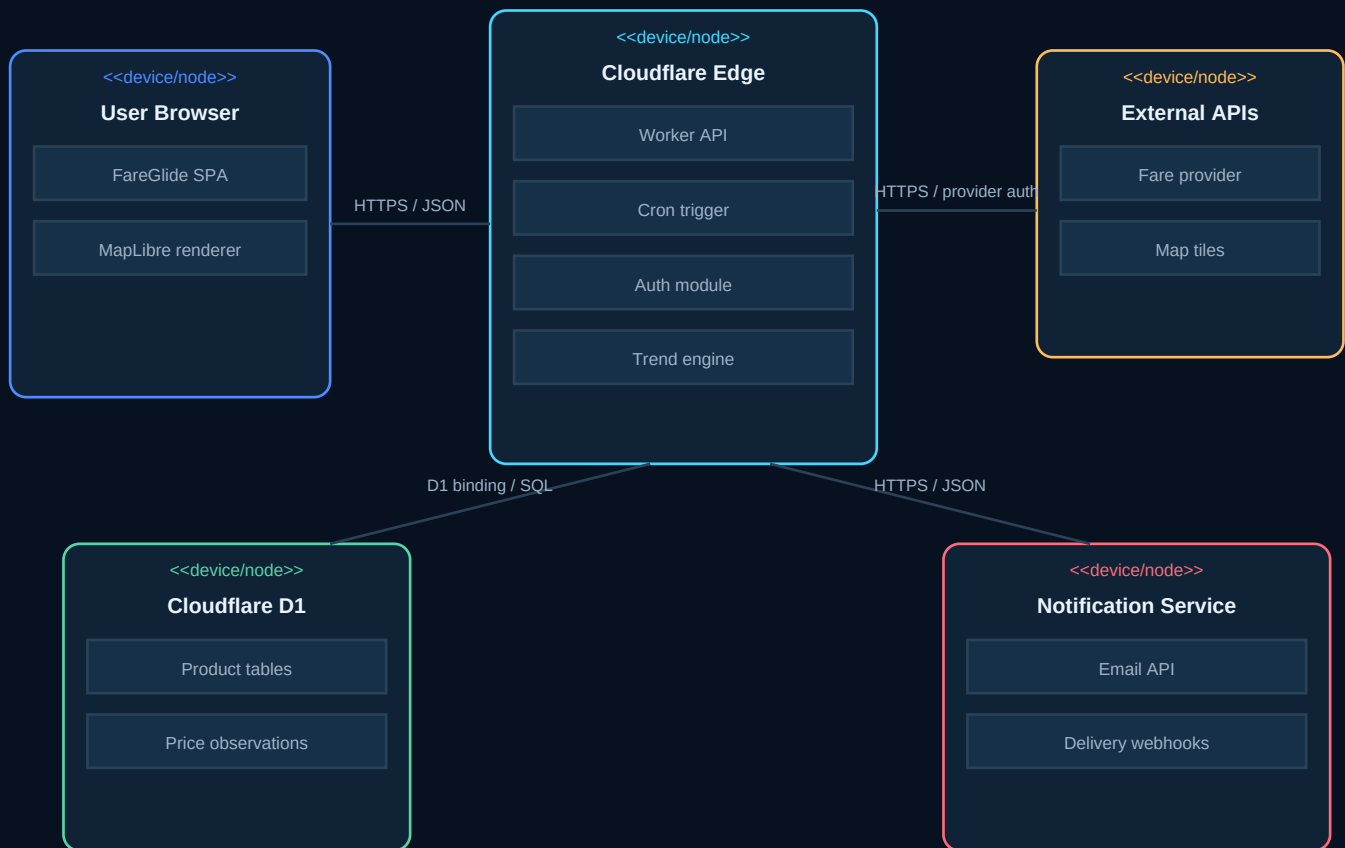


Modeling choice

Many users can attach RouteWatch records to one NormalizedSearchKey. PriceObservation rows therefore scale with unique searches, not directly with the number of users.

6. UML deployment diagram - planned hosted target

The first release runs as a local modular monolith. This deployment view records the intended hosted mapping so local interfaces remain portable.

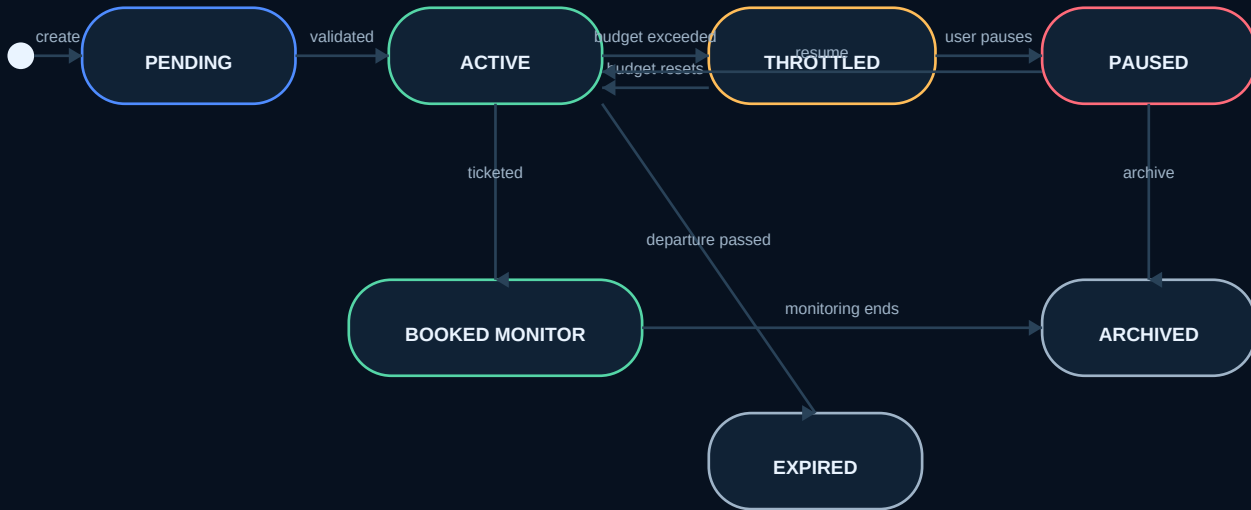


Deployment constraints

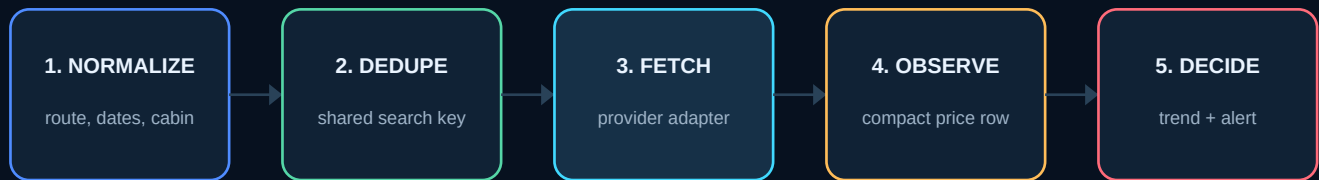
- The React application and map renderer execute in the user's browser.
- Authentication, provider calls, scheduled work, and trend evaluation execute at the edge.
- D1 stores both product and observation tables with separate retention and indexing policies.
- External integrations communicate over HTTPS and remain replaceable.

UML state machine - route watch

A route watch moves through explicit states so pausing, throttling, expiry, and archival never depend on ambiguous flags.



7. Search and observation pipeline



One provider request can refresh many users' identical watches

Normalized search key

A search key represents the dimensions that can materially change the fare: origin, destination, outbound date, return date, trip type, cabin, passengers, stop limit, market, and currency. Identical keys share one refresh even when many accounts watch them.

Example

SEA | NRT | 2026-11-04 | 2026-11-18 | RT | ECON | 1ADT | 1STOP | US | USD

Price observation

Field group	Examples
Identity	search_key, provider, observed_at
Price	currency, lowest_total, lowest_nonstop, median_offer
Itinerary	carrier, flight numbers, stops, duration, departure times
Quality	offer count, freshness, provider status, confidence
Booking	deeplink or provider reference; revalidate before handoff

Retention rule

Keep compact derived observations for charts. Do not assume that entire third-party payloads or bookable offers may be retained indefinitely; provider terms must be reviewed before production.

8. User and background flows

Flow	Steps	Failure behavior
Map exploration	Load dark tiles and airport catalog; show stored indicative observations	Never issue live fare searches merely because the user pans
Manual search	Normalize -> cache check -> provider fetch -> save observation -> show results	Return last known observation with freshness label
Save watch	Authenticate -> validate watch -> attach to shared search key	Keep watch pending if the provider budget is exhausted
Scheduled refresh	Select due keys -> cache/lock/budget check -> fetch once per key -> store observation	Backoff, record error, disable adapter on CAPTCHA/block
Alert	Compare against rolling baseline -> apply threshold and cooldown -> enqueue email	Retain undelivered alert and retry without duplicating
Booked watch	Record paid fare and eligibility -> monitor meaningful drops -> suggest verification	Never promise reimbursement; stop when ineligible or departed

Alert decision

- Require a minimum number of observations before describing a trend.
- Trigger only when both a percentage and an absolute-dollar threshold are met.
- Compare against a rolling median rather than the immediately previous sample.
- Use per-watch cooldowns and a daily digest to prevent notification fatigue.
- Label every alert with observed time, provider, and data freshness.

Suggested defaults

Condition	Default
Meaningful drop	At least 12% and \$40 below 14-day median
Meaningful spike	At least 18% and \$60 above 14-day median
New low	Lowest observation in the watch's retained history
Cooldown	24 hours per watch and alert type

Adaptive refresh policy

Time before departure	Automatic cadence
More than 60 days	Once per day
30 to 60 days	Twice per day
7 to 30 days	Every 6 hours
Less than 7 days	Every 6 hours for the local prototype
After departure	Stop and archive the watch

Each watch receives timing jitter. Manual refresh uses the same cache, per-route cooldown, concurrency lock, and daily request ceiling; it never bypasses safety controls.

9. Post-booking monitoring

Once ticketed, ordinary tracking stops. Eligible watches may enter BOOKED_MONITORING. The feature looks only for a meaningful comparable price drop; a later increase does not affect an issued ticket.

Window	Cadence	Purpose
First 24 hours	Every 6 hours	Surface a possible penalty-free cancellation and rebooking opportunity
After 24 hours	Once daily when eligible	Find a drop that may justify change, cancellation, or airline credit
Ineligible or departed	Stopped	Avoid spending requests when no action is available

Required booking context

- Amount paid and booking time.
- Airline and booking channel.
- Fare type and known change or refund eligibility.
- Known change fees and credit restrictions.

Alert language

FareGlide says potential rebooking opportunity - verify with the airline. It never states that reimbursement is owed, and it compares savings after known fees.

10. Free-first deployment

Layer	Choice	Why
Web	Existing React / TypeScript app	Already scaffolded; supports dashboard and map
Map	MapLibre + OpenFreeMap Dark	No key; customizable dark vector style
Airport catalog	OurAirports snapshot	Public-domain coordinates and codes
API	Cloudflare Worker	Edge deployment and secret isolation
Database	Cloudflare D1	SQLite semantics and useful free limits
Jobs	Cron Trigger + due-watch table	Simple scheduled collection without a dedicated server
Charts	Recharts	Already installed in the project
Email	Provider adapter	Avoid locking alerts to one vendor

Cost guardrails

- Ten active watches per local profile initially.
- Adaptive cadence: 1/day, then 2/day, then every 6 hours as departure approaches.
- Global deduplication of identical search keys.
- Manual-search cache of 30 to 60 minutes.
- Provider-specific daily budgets, randomized timing, exponential backoff, and a global kill switch.
- Reserve SerpApi quota for exact saved routes and use the local scraper only for fallback or sampled validation.
- Automatically disable the local Google adapter when CAPTCHA or access blocking is detected.
- When budget is exhausted, serve stored trends instead of failing the whole product.
- Do not enable Aviationstack or OpenSky polling until a feature demonstrably needs it.

Security and privacy

- Provider keys remain server-side and are never committed to GitHub.
- Account data is isolated by user ID; normalized search observations contain no personal data.
- Emails are stored only for authentication and notifications, with unsubscribe controls.
- Scraper output is treated as untrusted input and validated against a strict schema.

11. Delivery plan and decision gates

Phase	Deliverable	Exit criterion
0. Provider spike	Compare SerpApi, guarded local Google, Scrape Badger, and Amadeus	Coverage, stability, latency, terms, fields, and real cost documented
1. Experience prototype	Dark globe, airport search, route selection, mock trend chart	Primary workflow is recognizable and usable
2. Data MVP	Provider gateway, normalized keys, cache, observations, budgets	Repeat searches reuse stored data and survive provider errors
3. Personal MVP	Accounts, saved watches, dashboard, scheduled refresh	A user can return and see accumulated history
4. Alert MVP	Trend engine, outbox, email, cooldowns	Synthetic and real price changes produce one explainable alert
5. Enrichment	Optional status and aircraft overlays	Only after pricing workflow is stable

Decision gates

- Gate A - Scraping permission and reliability: the Google adapter remains local-only; public use requires a fresh legal and provider review.
- Gate B - Official access: pursue Skyscanner; do not depend on Kiwi Tequila without an invitation.
- Gate C - Storage rights: confirm what derived price history may be retained and displayed.
- Gate D - Unit economics: estimate searches per active watch before broad public launch.

Recommended immediate action

Build Phase 0 and Phase 1 together: a dark-map prototype backed by simulated history, SerpApi as the primary collector, and an isolated local Google Flights fallback. Exercise collection only through route creation, permitted manual refresh, and the adaptive scheduler.

Sources

Scrape Badger Google Flights scraper

<https://apify.com/scrape.badger/google-flights-scraper>

Scrape Badger pricing

<https://scrapebadger.com/pricing>

SerpApi Google Flights API

<https://serpapi.com/google-flights-api>

SerpApi pricing

<https://serpapi.com/pricing>

Google Terms of Service

<https://policies.google.com/terms?hl=en-US>

Kiwi.com partnership change

<https://media.kiwi.com/articles-and-interviews/better-for-business-kiwi-com-takes-a-new-approach-to-partnerships/>

Aviationstack pricing

<https://aviationstack.com/pricing>

OpenSky API

<https://opensky-network.org/data/api>

OpenSky FAQ and rate limits

<https://opensky-network.org/about/faq>

Skyscanner authentication

<https://developers.skyscanner.net/docs/getting-started/authentication>

Skyscanner flight API FAQ

<https://developers.skyscanner.net/docs/faqs>

OpenFreeMap quick start

https://openfreemap.org/quick_start/

Cloudflare Workers pricing

<https://developers.cloudflare.com/workers/platform/pricing/>

Cloudflare D1 pricing

<https://developers.cloudflare.com/d1/platform/pricing/>

Notes

Pricing and access terms were reviewed in August 2026 and may change. The supplied YouTube page could not be transcribed through the available page reader, so the Scrape Badger assessment uses the vendor's current product pages and API listing directly.